Abstraktní třídy

David Marek

Co je to abstraktní třída?

Abstraktní třída je speciální druh třídy, ze které **nemůžete vytvořit objekt** (instanci) pomocí klíčového slova `new`. Slouží výhradně jako neúplná šablona pro ostatní třídy.

Představte si, že stavíte aplikaci pro evidenci zvířat. Můžete mít "Psa" nebo "Kočku", ale dává smysl mít v systému objekt, který je prostě jen "Zvíře"? Asi ne. Zvíře je v tomto případě příliš obecný pojem – je to **abstrakce**.

Deklarace abstraktních prvků

Abstraktní třída se definuje klíčovým slovem `abstract`. Může obsahovat jak běžné metody s tělem, tak i **abstraktní metody**, které tělo nemají a potomek je **musí** povinně implementovat.

```
// Abstraktní třída - nelze vytvořit 'new Animal()'
public abstract class Animal
{
    // Abstraktní metoda - nemá tělo, potomek JI MUSÍ přepsat
    public abstract void MakeSound();

    // Běžná metoda - potomci ji zdědí tak, jak je
    public void Sleep()
    {
        Console.WriteLine("Sleeping...");
    }
}

// Potomek MUSÍ implementovat MakeSound
public class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Woof!");
    }
}
```

Rozdíl: Abstract vs. Virtual

Obě klíčová slova umožňují polymorfismus, ale s jinými pravidly:

Vlastnost	Abstraktní metoda (`abstract`)	Virtuální metoda (`virtual`)
Má tělo (kód)?	**Ne**	**Ano**
Musí se přepsat?	**Ano** (v prvním neabstraktním potomkovi)	**Ne** (může se použít původní)
Kde může být?	Pouze v abstraktní třídě	V jakékoliv třídě

```
public abstract class Vehicle
{
    // MUSÍ se přepsat - neexistuje "univerzální" startování
    public abstract void StartEngine();

    // MŮŽE se přepsat - většina aut troubí stejně, ale někdo může chtít jiný zvuk
    public virtual void Honk()
    {
        Console.WriteLine("Beep!");
    }
}
```

Proč abstraktní třídy používat?

1. **Vynucení kontraktu:** Máte jistotu, že každý potomek (např. každé auto) bude mít metodu `StartEngine()`.
2. **Zamezení chybám:** Nemůžete omylem vytvořit "obecný" objekt, který nemá definované základní chování.
3. **Sdílení kódu:** Na rozdíl od rozhraní (interface) mohou abstraktní třídy obsahovat i hotový kód, který se dědí (např. metoda `Sleep()`).

Úkol k procvičení: Geometrické tvary

Vaším úkolem je vytvořit systém pro výpočet obsahu a obvodu různých tvarů.

1. **Abstraktní třída `Shape`:** Obsahuje dvě abstraktní metody typu `double`:
   - `CalculateArea()`
   - `CalculatePerimeter()`
2. **Třída `Circle` (dědí ze `Shape`):**
   - Obsahuje soukromé pole `_radius`.
   - Konstruktor nastaví poloměr.
   - Vzorce:
     - Obsah: $S = \pi \cdot r^2$
     - Obvod: $O = 2 \cdot \pi \cdot r$
     - *(Tip: V C# pro hodnotu Pi použijte `Math.PI`)*
3. **Třída `Rectangle` (dědí ze `Shape`):**
   - Obsahuje pole `_width` a `_height`.
   - Konstruktor nastaví obě hodnoty.
   - Vzorce:
     - Obsah: $S = a \cdot b$
     - Obvod: $O = 2 \cdot (a + b)$

V Program.cs: Vytvořte seznam `List<Shape>`, vložte do něj jeden kruh a jeden obdélník. Pomocí cyklu `foreach` vypište pro každý tvar jeho vypočítaný obsah a obvod.

```
List<Shape> shapes = [
    new Rectangle(3.0, 2.0),
    new Circle(4.0)
];

// TODO: Vypsání obsahů a obvodů v foreach
```